INTERNSHIP REPORT OF

WEB DEVELOPMENT INTERNSHIP

Submitted To



SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

IILM UNIVERSITY, GREATER NOIDA, U.P.

In partial fulfilment of the requirement of degree of

B.TECH (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

PROJECTS UNDERTAKEN

1. Interactive Navigation Menu
2. Stopwatch Web Application
3. Tic-Tac-Toe Game

By

Name of Student: DRISHTI THAKUR

Roll Number of Student: 25SCS1003002554

Program & Semester: B.Tech (AI & ML), 3rd Semester

Organization: Prodigy Infotech

Batch: 2025-2029

CANDIDATE'S DECLARATION

I, Drishti Thakur, do hereby declare that the internship report titled "Web Development Internship" has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in Artificial Intelligence & Machine Learning.

The work presented in this report is based on the practical learning and projects completed during my internship with Prodigy Infotech. All the references used in preparing this report have been properly acknowledged. I affirm that this report is my own work and has not been submitted to any other institute or university for the fulfilment of any degree or diploma requirement.

I further declare that I shall be solely responsible for any kind of copyright violation arising from the contents of this report.

Signature: _____  _____

Student Name: Drishti Thakur

Roll No.: 25SCS1003002554

ACKNOWLEDGEMENT

I am using this opportunity to express my gratitude to everyone who supported me throughout my internship. I am thankful for their guidance, valuable suggestions, constructive feedback, and continuous encouragement during the completion of this work.

I would like to express my sincere gratitude to Prodigy Infotech for providing me with the opportunity to participate in the Web Development Internship and gain practical exposure to front-end web technologies. The internship provided me with an opportunity to understand the practical implementation of HTML, CSS, and JavaScript concepts through hands-on projects.

I am also thankful to IILM University, Greater Noida, and the School of Computer Science and Engineering for providing the academic environment and support required to complete this internship and prepare this report.

I would also like to express my gratitude to my faculty members, mentors, friends, and family for their support and encouragement throughout the internship.

Student Name: DRISHTI THAKUR

Roll No.: 25SCS1003002554

TABLE OF CONTENTS

1. Candidate's Declaration
2. Acknowledgement
3. Project Description
    3.1 Introduction
    3.2 Organization Profile
    3.3 Problem Statement
    3.4 Project Objectives
    3.5 Scope of the Project
    3.6 Technologies and Tools Used
    3.7 System Architecture
    3.8 Methodology
    3.9 Testing and Results
    3.10 Challenges Faced
    3.11 Learning Outcomes
    3.12 Expected Outcomes
4. Bibliography / References

3. PROJECT DESCRIPTION

3.1 Introduction

Web development is one of the most widely applied areas of computer science, covering the structure, styling, and behaviour of interactive interfaces that run directly in the browser. Front-end development in particular — HTML for structure, CSS for presentation, and JavaScript for interactivity — forms the foundation on which nearly every web-based product is built.

This internship was undertaken with Prodigy Infotech in the domain of Web Development. The main purpose of the internship was to gain practical, hands-on experience with core front-end technologies by independently building complete, working browser-based applications rather than studying the concepts in isolation.

Three projects were completed during the internship and are presented in detail in this report: an Interactive Navigation Menu, a Stopwatch Web Application, and a Tic-Tac-Toe Game. Each project targets a different core skill — responsive, state-driven UI design; precise timing and data tracking; and game-state logic with simple decision-making — while relying on the same three underlying technologies: HTML, CSS, and JavaScript, with no external frameworks.

Together, these projects provided practical exposure to DOM manipulation, event handling, CSS transitions and responsive layout, accurate time-based computation, and rule-based decision logic, along with the surrounding practices of version control, testing, and documentation.

3.2 Organization Profile

Prodigy Infotech is a Mumbai-based EdTech and IT services organization that offers 100% remote, task-based virtual internships for students and freshers across domains such as Web Development, Data Science, Machine Learning, Android Development, and Cybersecurity.

Rather than delivering theory-heavy modules, Prodigy Infotech assigns interns a series of independent, project-style tasks to complete on their own, with the goal of building a practical, resume-ready portfolio by the end of the program. A certificate of completion is awarded once the assigned tasks are finished.

The Web Development Internship provided practical exposure to building interactive, browser-based interfaces using HTML, CSS, and JavaScript, and encouraged learning through direct implementation rather than theoretical study alone — including planning, building, testing, debugging, and documenting each project independently.

3.3 Problem Statement

Modern websites are expected to be interactive, responsive, and visually polished — not merely static pages of text and images. Achieving this requires practical knowledge of how browsers render and update content, how user interactions such as clicks, scrolling, and hovering can be captured and responded to, and how visual state can be changed smoothly using CSS.

The main problem addressed during this internship was to design and implement three distinct, self-contained front-end applications that each demonstrate a different aspect of browser-based interactivity, using only core web technologies:

- Interactive Navigation Menu: Build a fixed-position navigation bar that visually responds to scrolling and hovering, and remains consistent across multiple pages.
- Stopwatch Web Application: Build a timing tool capable of accurately starting, pausing, resetting, and recording lap times without drifting over time.
- Tic-Tac-Toe Game: Build a two-player browser game with full win/draw detection and an optional computer opponent capable of making sensible moves.

Each project required translating a familiar, everyday concept — a website's navigation bar, a physical stopwatch, a paper-and-pencil game — into a fully working, rule-driven web application.

3.4 Project Objectives

The major objectives of the internship projects were:

- To understand how HTML, CSS, and JavaScript work together to build interactive interfaces.
- To gain practical experience with DOM manipulation and browser event handling.
- To implement scroll-based and hover-based UI state changes using CSS transitions and JavaScript class toggling.
- To design a responsive layout that adapts cleanly from desktop to mobile screens.
- To understand accurate time measurement in JavaScript using performance.now() and requestAnimationFrame().
- To implement data-tracking features, such as lap recording, with derived values including splits and totals.
- To design rule-based game logic, including win and draw detection across all possible outcomes.
- To implement a simple, explainable decision-making opponent (heuristic AI) without relying on external libraries.
- To practise structuring, testing, and debugging small front-end projects independently.
- To maintain project code using Git and prepare it for publishing on GitHub.
- To gain practical project development experience applicable to future front-end and full-stack development work.

3.5 Scope of the Project

The scope of this internship covers the practical implementation of three front-end web applications.

Project 1: Interactive Navigation Menu

The first project is a fixed navigation bar, shared identically across multiple pages, that changes its background, shadow, and link styling in response to scrolling and hovering.

The project can be extended for:

- Multi-page websites and portfolios
- Landing pages requiring a persistent call-to-action bar
- Dashboards and admin interfaces
- Any site needing consistent, state-aware navigation

Project 2: Stopwatch Web Application

The second project is a browser-based stopwatch supporting start, pause, reset, and lap recording, with automatic highlighting of the fastest and slowest laps.

The system can be used for:

- Timing everyday tasks and workouts
- Practising accurate time-based JavaScript programming
- Demonstrating requestAnimationFrame-driven UI updates
- As a foundation for more advanced timing tools, such as countdowns and interval timers

Project 3: Tic-Tac-Toe Game

The third project is a two-mode Tic-Tac-Toe game supporting both two-player and single-player (vs. computer) play, with full win-line detection and score tracking.

The system can be used for:

- Demonstrating rule-based game-state management
- Learning simple, heuristic-based decision-making, as opposed to exhaustive search algorithms such as minimax
- Practising DOM-driven rendering of dynamic game state
- As a starting point for more advanced browser-based games

The three projects are educational, portfolio-style implementations intended to demonstrate core front-end concepts; they are not intended as production-scale commercial products.

3.6 Technologies and Tools Used

Programming Languages

HTML5, CSS3, and JavaScript (ES6+) were used as the core technologies for all three projects, with no external front-end frameworks or libraries, so that the underlying browser behaviour could be understood and controlled directly.

Fonts and Assets

Google Fonts (Space Grotesk, Inter, Caveat, and JetBrains Mono) were used across the three projects to give each interface a distinct, purpose-fitting visual identity: a technical, geometric pairing for the navigation menu, a monospaced chronograph style for the stopwatch, and a handwritten chalkboard style for the Tic-Tac-Toe game.

Browser APIs

- DOM APIs (querySelector, addEventListener, classList) were used throughout for reading and reacting to user interaction.
- The window scroll event and window.scrollY were used to drive the navigation menu's scroll-based state.
- performance.now() and requestAnimationFrame() were used in the stopwatch for frame-accurate timing, in place of the less reliable setInterval().
- localStorage was used in the Tic-Tac-Toe game to persist the score across page reloads.

Development Tools

Visual Studio Code was used as the development environment for writing, running, and debugging all three projects. Git and GitHub were used for project version control and for hosting the completed source code.

3.7 System Architecture

Overall Architecture

All three projects follow the same general interaction pattern common to client-side web applications:

User Interaction → Event Listener → State Update → DOM / CSS Update → Rendered Result

Project 1: Navigation Menu Architecture

User scrolls or hovers

↓

JavaScript scroll/hover event listener fires

↓

A CSS class (e.g. "is-scrolled") is toggled on the nav element

↓

CSS transitions animate the background, shadow, and link colour

↓

Updated navigation bar is rendered

As the user scrolls the page, a scroll event listener checks the current scroll position (window.scrollY) against a fixed threshold. Once the threshold is passed, JavaScript adds an is-scrolled class to the navigation bar; CSS transitions then animate the change in background colour, blur, and shadow smoothly rather than switching instantly. Hovering over a link is handled independently through CSS :hover, animating an underline into view.

Project 2: Stopwatch Architecture

User clicks Start

↓

performance.now() timestamp recorded

↓

requestAnimationFrame loop begins

↓

Elapsed time recalculated on every animation frame

↓

Time display updated (mm:ss.cc)

↓

Lap click records current split and total

↓

Lap list re-rendered with best/worst split highlighted

Rather than using setInterval(), which can drift over long periods, the stopwatch records a starting timestamp with performance.now() and recalculates the elapsed time on every animation frame using requestAnimationFrame(). This keeps the displayed time accurate regardless of how the browser schedules other work. Pausing stores the accumulated elapsed time; resuming starts a new timing segment from the current total.

Project 3: Tic-Tac-Toe Architecture

User clicks a cell

↓

Move validity checked (cell empty, game not over)

↓

Board state array updated

↓

Win/draw check run across all 8 winning lines

↓

DOM updated with the new mark

↓

If "Vs Computer" mode: heuristic AI selects a move (win → block → center → corner → random)

↓

Board and status re-rendered

The 3×3 board is represented internally as a flat array of nine cells. Every move updates this array first, and a single result-checking function then tests the array against all eight possible winning lines — three rows, three columns, and two diagonals — plus a full-board check for a draw. Only after the state is confirmed valid does the corresponding DOM cell get updated with the player's mark, keeping the visible board and the internal game state consistent at all times.

3.8 Methodology

The internship projects were completed using a step-by-step development methodology, repeated for each project.

Project 1: Interactive Navigation Menu

- Structure Planning: The navigation bar's HTML structure (brand, links, mobile toggle) was planned and written first, then duplicated identically across all pages of the demo site.
- Styling: CSS was written for the default (transparent) and scrolled (solid) states, along with hover transitions and a responsive mobile layout.
- Scroll Behaviour: A JavaScript scroll event listener was added to toggle the "is-scrolled" class once the page passed a set scroll threshold.
- Mobile Menu: A hamburger toggle button was implemented to show and hide the link list on small screens.
- Active-link Highlighting: JavaScript was used to detect the current page and highlight the matching navigation link.
- Testing: The menu was tested across page loads, scroll positions, hover states, and screen sizes.

Project 2: Stopwatch Web Application

- Timing Logic: The core timing logic was implemented first, using performance.now() to track elapsed time independent of the display.
- Display Loop: A requestAnimationFrame loop was added to continuously update the on-screen time while the stopwatch is running.
- Start / Pause / Reset: Button handlers were implemented to start a new timing segment, pause and store the accumulated time, and reset the stopwatch, available only once paused.
- Lap Tracking: Lap recording was added, storing both the split (time since the last lap) and the running total for each lap.
- Highlighting: The fastest and slowest laps were automatically identified and highlighted once two or more laps existed.
- Testing: The stopwatch was run over multiple sessions to confirm accurate, drift-free timing and correct lap calculations.

Project 3: Tic-Tac-Toe Game

- Board Representation: The 3×3 board was represented as a flat nine-element array, simplifying win-checking and click handling.
- Click Handling: Event listeners were attached to each cell to validate and register moves.
- Win/Draw Detection: A function was written to check the board against all eight winning lines and a full-board draw condition.
- Two-Player Mode: Turn-based alternation between X and O was implemented for local two-player play.
- Computer Opponent: A heuristic AI was added for single-player mode — it checks for an immediate winning move, then a blocking move, then defaults to the centre, a corner, or a random empty cell.

- Scoring and Persistence: Win/draw counts were added and stored in localStorage so they persist between visits.
- Testing: Each mode was played through multiple rounds to confirm correct win detection, AI behaviour, and score tracking.

3.8.1 Workflow of the Internship Projects

The complete workflow followed during the projects can be summarised as:

Requirement Understanding
↓
Structure & Layout Planning
↓
HTML / CSS Implementation
↓
JavaScript Logic Implementation
↓
Testing and Debugging
↓
Cross-Device / Cross-Browser Check
↓
Documentation
↓
Version Control (Git) & GitHub Publishing

This workflow helped maintain a systematic and repeatable approach across all three projects.

3.9 Testing and Results

Interactive Navigation Menu — Results

The navigation menu successfully switched between its transparent and solid states based on scroll position, with smooth CSS-driven transitions rather than an abrupt change. Hover interactions animated correctly, and the layout adapted properly to smaller screen widths through the mobile menu.



Figure 3.1: Navigation bar at the top of the page (transparent state).

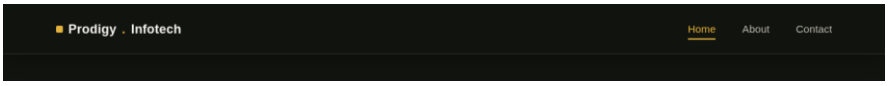


Figure 3.2: Navigation bar after scrolling (solid background with shadow).

Stopwatch — Results

The stopwatch accurately tracked elapsed time across multiple start/pause cycles without drifting, and correctly calculated per-lap splits and running totals. The fastest and slowest laps were correctly identified once multiple laps were recorded.

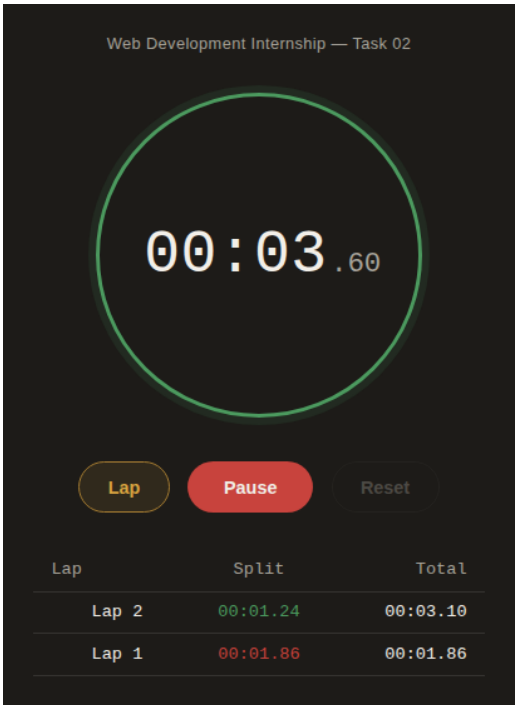


Figure 3.3: Stopwatch running, with two recorded laps and split highlighting.

Tic-Tac-Toe — Results

The game correctly detected wins across all eight possible lines and correctly identified draws when the board filled without a winner. In "Vs Computer" mode, the heuristic opponent reliably took winning moves when available and blocked the player's winning moves otherwise.

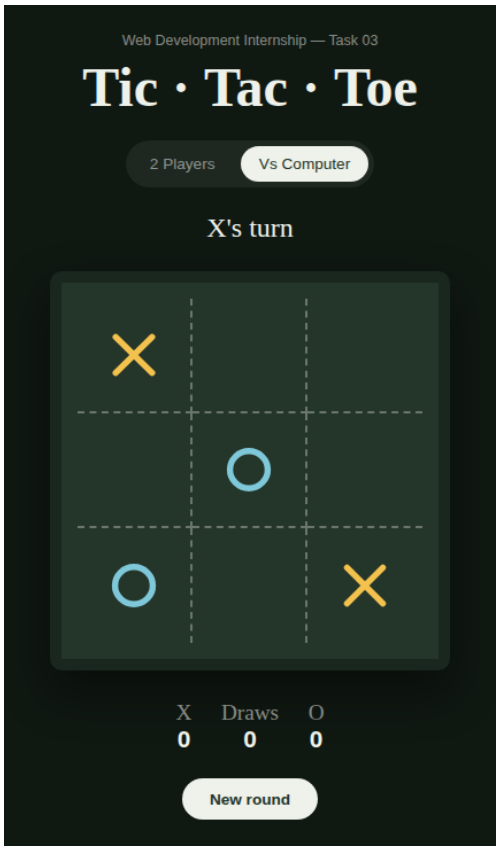


Figure 3.4: Tic-Tac-Toe game in progress against the computer opponent.

3.10 Challenges Faced

During the implementation of the projects, several practical challenges were encountered.

- Timer Drift: An initial implementation using setInterval() for the stopwatch showed slight timing drift over longer sessions; this was resolved by switching to performance.now() with a requestAnimationFrame loop.
- Consistent Navigation Across Pages: Keeping the navigation bar's markup, styling, and behaviour identical across multiple pages required careful reuse of the same HTML, CSS, and JS files rather than rebuilding the component per page.
- Designing a Balanced Computer Opponent: Implementing an AI opponent that plays sensibly without the added complexity of a full minimax search required designing a simple, explainable heuristic (win → block → center → corner → random).
- Responsive Behaviour: Ensuring all three interfaces remained usable on smaller screens required additional testing and CSS adjustments, particularly for the navigation menu's mobile layout.
- Cross-browser Consistency: Minor differences in how browsers handle CSS transitions and backdrop-filter blur required small adjustments for consistent visual behaviour.

3.11 Learning Outcomes

The internship provided practical knowledge in several areas:

- DOM manipulation and event handling
- CSS transitions, responsive design, and layout techniques

- Accurate time measurement using performance.now() and requestAnimationFrame()
- State management in vanilla JavaScript
- Rule-based logic design, including win and draw detection
- Heuristic decision-making for a simple AI opponent
- Use of localStorage for client-side persistence
- Git and GitHub for version control
- Debugging and iterative testing of front-end code
- Project documentation and presentation

The projects also helped in understanding the difference between purely CSS-driven interactivity, such as hover states and transitions, and JavaScript-driven interactivity, such as state tracking, timing, and decision logic, and when each is the more appropriate tool.

3.12 Expected Outcomes

The major outcomes of the internship are:

- Practical understanding of core front-end web development concepts.
- Ability to independently plan, build, and test browser-based applications using HTML, CSS, and JavaScript.
- Experience implementing scroll- and state-driven UI behaviour.
- Practical experience with precise, frame-accurate timing in JavaScript.
- Experience designing rule-based game logic and simple heuristic decision-making.
- Improved debugging and problem-solving skills for front-end code.
- Experience maintaining project code using Git and preparing it for GitHub.
- A foundation for further learning in front-end frameworks such as React, back-end development, and full-stack web development.

4. BIBLIOGRAPHY / REFERENCES

1. Prodigy Infotech, “Web Development Internship Program,” Internship Learning and Task Resources, 2026.
2. Mozilla Developer Network (MDN), “HTML, CSS, and JavaScript Documentation,” developer reference for web technologies.
3. MDN Web Docs, “Window: requestAnimationFrame() method,” documentation for animation-frame scheduling.
4. MDN Web Docs, “Performance: now() method,” documentation for high-resolution time measurement.
5. MDN Web Docs, “Window.localStorage,” documentation for client-side data persistence.
6. Google Fonts, “Space Grotesk, Inter, Caveat, and JetBrains Mono,” typefaces used across the three projects.
7. Git, “Git Documentation,” documentation for version control.
8. GitHub, “GitHub Documentation,” documentation for repository management and hosting.
9. W3C, “HTML and CSS Specifications,” World Wide Web Consortium technical standards.